COLLEGE OF ENGINEERING, GUINDY

Chennai - 600025

DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY - IST

Assignment Report on

Implementation of RPC on AWS Cloud

in

DISTRIBUTED SYSTEMS AND COMPUTING – IT23601

Submitted by

Mohan Vishnu Kumar – 2023115026

Implementation of Remote Procedure Call (RPC) in a Cloud Environment (AWS)

1. Introduction

Distributed systems allow multiple computers to work together and communicate over a network to achieve a common goal. One of the most important communication mechanisms in distributed systems is **Remote Procedure Call (RPC)**.

RPC enables a client program to call a function or procedure that exists on a remote server as if it were a local function. The complexity of networking, data transmission, and response handling is hidden from the user.

In this assignment, I have implemented an RPC-based application using Python and deployed it on a cloud environment (AWS EC2). The client can remotely invoke procedures on the server, and the server processes the request and returns the result.

2. Objective of the Experiment

The main objectives of this experiment are:

1. To understand the working of Remote Procedure Call (RPC).
2. To implement an RPC-based client-server application.
3. To host the server on a cloud platform.
4. To enable remote access of server functions by the client.
5. To handle errors gracefully.
6. To observe real-time distributed communication.

3. Tools and Technologies Used

Component	Description
Programming Language	Python 3
RPC Library	xmlrpc
Cloud Platform	Amazon Web Services (AWS EC2)

Operating System (Server)	Ubuntu 22.04
Client OS	Windows
Text Editor	VS Code / Nano
Protocol Used	HTTP

4. System Architecture

The system consists of two main components:

1. RPC Server

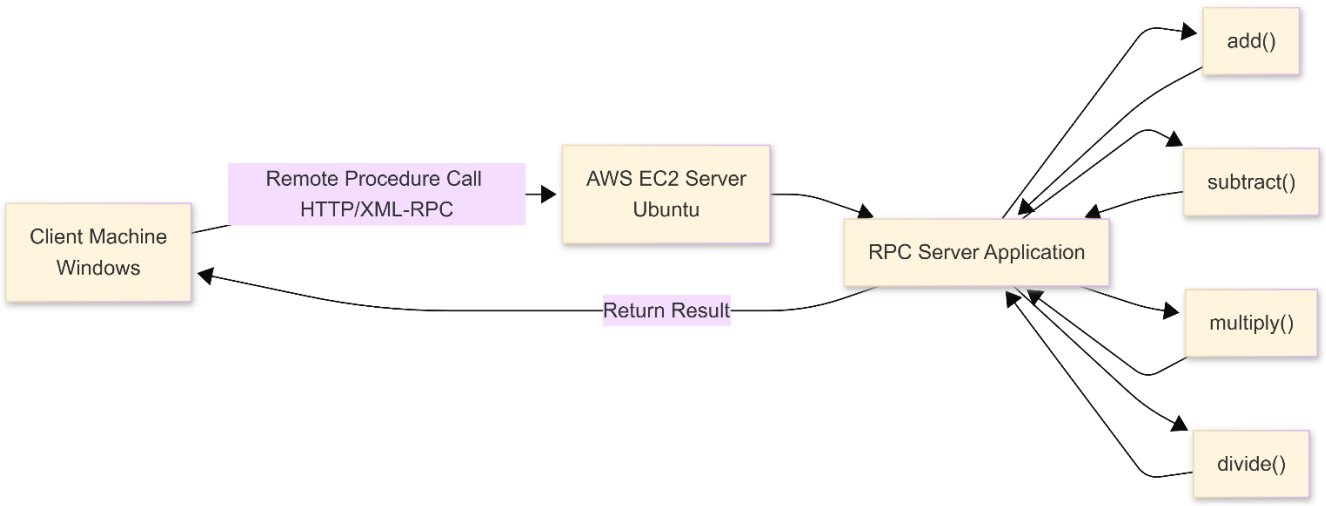
The server defines and exposes remote procedures such as:

1. add()
2. subtract()
3. multiply()
4. divide()

These procedures are registered using the XML-RPC framework and made accessible over the internet.

2. RPC Client

The client connects to the remote server using its public IP address and calls the server's functions. The result is received and displayed on the client machine.



5. Working of RPC in This Project

1. The server is started on an AWS EC2 instance.
2. The server listens on port 8000.
3. The client uses the server's public IP address to connect.
4. The client invokes remote procedures.
5. The server executes the procedure.
6. The result is returned to the client.
7. The client displays the output.

6. Implementation Details

6.1 Server Side Implementation

The server defines all the required remote functions. These functions are registered using the SimpleXMLRPCServer module.

Error handling is implemented to avoid crashes and invalid inputs.

Server Features:

- Accepts multiple remote calls
- Performs arithmetic operations
- Handles division by zero
- Returns error messages for invalid inputs
- Runs continuously

Code:

```
from xmlrpc.server import SimpleXMLRPCServer

def add(a, b):
    print("Performing addition")
    return a + b

def subtract(a, b):
    print("Performing subtraction")
    return a - b

def multiply(a, b):
    print("Performing multiplication")
    return a * b
```

```
def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    print("Performing Division")
    return a / b

server = SimpleXMLRPCServer(("0.0.0.0", 8000))
print("RPC Server is running on port 8000...")

server.register_function(add, "add")
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")

server.serve_forever()
```

6.2 Client Side Implementation

The client connects to the remote server using its public IP address and port number. The client invokes the remote functions as if they were local.

Exception handling is implemented to manage connection errors and unexpected failures.

Client Features:

- Connects to cloud-hosted server
- Invokes remote procedures
- Displays results
- Handles connection failures

Code:

```
import xmlrpc.client

try:
    server = xmlrpc.client.ServerProxy("http://13.48.249.5:8000/")

    print("Addition:", server.add(5, 3))
    print("Subtraction:", server.subtract(10, 4))
    print("Multiplication:", server.multiply(6, 7))
    print("Division:", server.divide(20, 5))
    print("Division by zero:", server.divide(10, 0))

except ConnectionRefusedError:
    print("Error: Could not connect to the server.")
except Exception as e:
    print("Unexpected error:", str(e))
```

7. Cloud Deployment (AWS EC2)

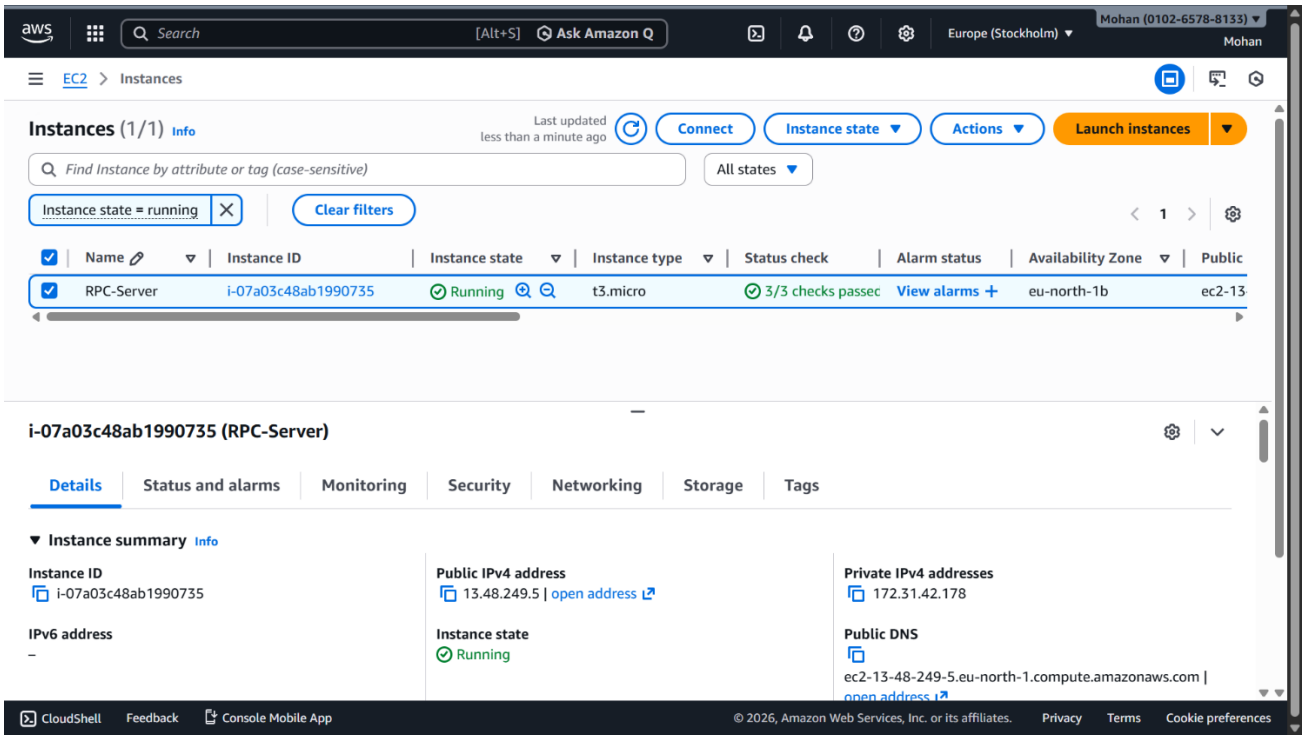
To make the system truly distributed, the RPC server was hosted on AWS EC2.

Steps Followed:

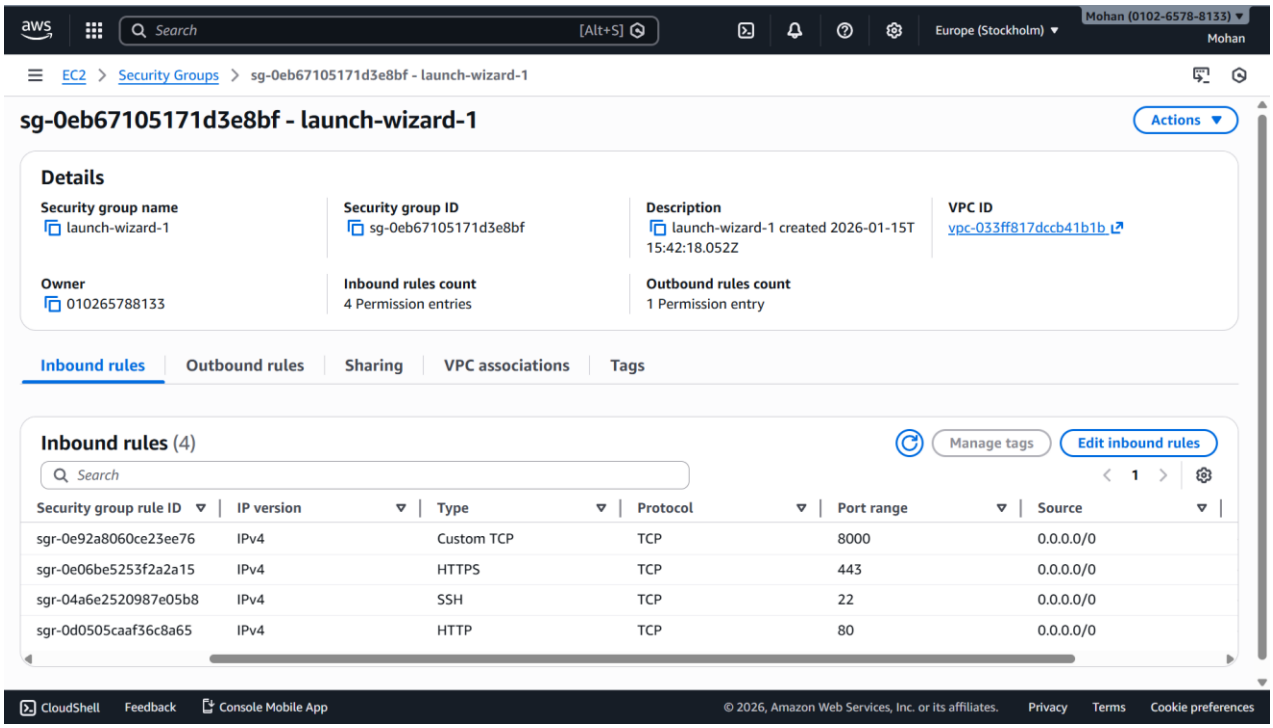
1. Created a free-tier EC2 instance on AWS.
2. Selected Ubuntu as the operating system.
3. Connected to the instance using SSH.
4. Installed Python.
5. Uploaded the RPC server code.
6. Opened port 8000 in the inbound rules.
7. Started the server.
8. Connected the client using the public IP.

This enabled real-time remote procedure execution across different machines.

AWS EC2 Instance:



Inbound Rule:



Starting the server through SSH:

```
PS C:\Users\curvy\Downloads> ssh -i rpc-key.pem ubuntu@13.48.249.5
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Jan 15 16:10:25 UTC 2026

  System load:  0.0               Temperature:           -273.1 C
  Usage of /:   26.4% of 6.71GB   Processes:             117
  Memory usage: 26%               Users logged in:       1
  Swap usage:   0%                IPv4 address for ens5: 172.31.42.178


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Thu Jan 15 15:51:57 2026 from 223.237.183.108
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-42-178:~$ python3 rpc_server.py
RPC Server is running on port 8000...
```

8. Error Handling

Proper error handling was implemented on both client and server sides.

Server-side:

- Handles invalid inputs
- Prevents division by zero
- Returns meaningful error messages

Client-side:

- Handles server unavailability
- Catches connection errors
- Displays friendly error messages

This ensures that the system does not crash and remains stable.

9. Output

The client successfully invoked remote procedures and received correct outputs from the cloud-hosted server.

Client-side Output:

```
PS C:\Users\curvy\Desktop\RPC-DS_Assignment1> python .\rpc_client.py
Addition: 8
Subtraction: 6
Multiplication: 42
Division: 4.0
Division by zero: Error: Division by zero
```

Server-side logs:

```
ubuntu@ip-172-31-42-178:~$ python3 rpc_server.py
RPC Server is running on port 8000...
Adding 5 and 3..
223.237.183.108 - - [15/Jan/2026 16:11:32] "POST / HTTP/1.1" 200 -
Subtracting 4 from 10..
223.237.183.108 - - [15/Jan/2026 16:11:32] "POST / HTTP/1.1" 200 -
Multiplying 6 with 7..
223.237.183.108 - - [15/Jan/2026 16:11:33] "POST / HTTP/1.1" 200 -
Dividing 20 with 5..
223.237.183.108 - - [15/Jan/2026 16:11:33] "POST / HTTP/1.1" 200 -
223.237.183.108 - - [15/Jan/2026 16:11:34] "POST / HTTP/1.1" 200 -
```